

1. Vision

Problem Statement The Indian legal system is burdened by millions of pending cases, complex colonial-era statutes, and rigid procedural opacity. Regular citizens face prohibitive costs and an intimidating linguistic barrier, as formal legal documentation is predominantly in English or highly formalized Hindi, while citizens converse in regional languages or code-mixed dialects like Hinglish. Consequently, marginalized populations are effectively priced out of justice and forced into unregulated dispute resolution mechanisms.

Why Current Legal Assistance Fails Current digital legal tools are largely keyword-based search engines designed for lawyers, not citizens. They fail to understand colloquial grievances, lack multilingual and voice capabilities, and provide raw statutory text rather than actionable, procedural advice. Existing generative AI models hallucinate case laws and lack the deterministic rigidity required for reliable legal counsel.

Goals To build a scalable, trustworthy, and multilingual AI Legal Assistant using Gemma 4 that empowers regular Indian citizens to describe their disputes in plain terms (including voice and Hinglish), understand their rights, and receive a clear, actionable roadmap for their next steps.

Non-Goals

- Providing binding legal representation or acting as a substitute for a licensed attorney in court.
- Automating judicial sentencing or replacing human mediators.
- Providing definitive mathematical calculations for complex tax or corporate mergers.

Target Users Regular Indian citizens, particularly those in rural or low-income demographics who cannot afford preliminary legal consultation, as well as paralegals and legal aid clinics seeking to scale their triage processes.

Target Use Cases

- Dispute resolution (e.g., tenant-landlord conflicts, consumer rights, wrongful termination).
- Procedural guidance (e.g., "How do I file an FIR?", "What are the steps for a mutual divorce?").
- Document understanding (e.g., explaining a received legal notice in plain language).

2. Complete System Architecture

Our system follows a Multi-Agent State Machine architecture orchestrated via LangGraph.

Core Components & Data Flow:

1. **Client Interface (Frontend):** React.js/Tailwind web app accepting text, voice, and document uploads.
2. **API Gateway:** FastAPI backend handling rate-limiting, authentication (JWT), and session management.
3. **Orchestrator Agent:** The central decision-maker. It maintains conversation memory (Redis) and routes user inputs. If the query is conversational, it responds directly. If it requires legal facts, it triggers the RAG pipeline.
4. **Data Ingestion & Pre-processing Services:**
 - *File Reader Tool:* Extracts text from uploaded PDFs/Images using EasyOCR and

- PDFMiner.
- *Query Normalization Service*: Detects Hinglish/code-mixed inputs and maps them to formal English/Hindi keywords.
5. **Two-Stage RAG Pipeline (Vector DB):**
 - *ChromaDB (Acts)*: Contains semantic summaries of overarching laws (e.g., Bharatiya Nyaya Sanhita).
 - *ChromaDB (Sections)*: Contains specific clauses.
 6. **Inference Engine (Gemma 4)**: The core reasoning LLM hosted via vLLM. It takes the retrieved, verified sections and the user query to formulate an IRAC-structured response.
 7. **Verification & Guardrail Agents:**
 - *Confidence Gate*: Verifies semantic overlap between retrieved context and query before generation.
 - *Citation Generator*: Forces metadata-driven provenance mapping.
 8. **Output Services**: Translates the final formal response back into the user's colloquial language and generates an audio response via Bhashini TTS.

3. AI Pipeline

The complete pipeline from user input to final answer executes in the following sequence:

1. **Language & Code-Mixed Detection**: Identifies the user's input language. If Roman Hindi (Hinglish) is detected, it flags the input for normalization.
2. **Hinglish Normalization**: A specialized pre-processor translates romanized code-mixed text into standard formal English/Hindi to ensure semantic compatibility with the vector database.
3. **Intent Classification & Clarification**: The Orchestrator evaluates the query. If key variables (e.g., dates, jurisdiction) are missing, the *Clarification Agent* halts the pipeline and asks the user a probing question.
4. **Keyword Generation**: Translates the colloquial complaint (e.g., "owner threw me out") into formal semantic search terms (e.g., "illegal eviction", "tenant rights").
5. **Hierarchical Retrieval (Knowledge Base Router)**:
 - *Stage 1*: Retrieves the top 3 relevant Acts based on the keywords.
 - *Stage 2*: Uses the Act IDs as metadata filters to retrieve the specific top 5 Sections.
6. **Relevance Verification**: A lightweight cross-encoder checks if the retrieved sections actually answer the query. If the confidence is below 0.7, it triggers a retry or escalates to a human fallback.
7. **Legal Dictionary Enhancement**: Scans the retrieved sections for archaic Indian legal jargon (e.g., Urdu/Persian terms common in land records) and appends modern definitions.
8. **Gemma 4 Generation (IRAC Reasoning)**: Gemma 4 generates the response strictly following the Issue, Rule, Application, Conclusion framework.
9. **Citation Generator**: Appends exact statutory links (e.g., "BNS Section 302") to the generated claims.
10. **Translation & Speech Output**: The final formal output is simplified, translated into the

user's native tongue, and synthesized into an audio file.

4. Research-backed Components

Component	Why it exists	Source Paper	Why we adopted it	Implementation
Multi-Agent Orchestrator	Bypasses expensive RAG for casual chat, reducing latency and cost.	[1] (MINA)	Ensures the system is scalable and doesn't waste compute on follow-up clarifications.	LangGraph state machine with conditional routing edges.
Hierarchical RAG (Two-Stage)	Prevents "flattening" errors where models mix unrelated statutes from different domains.	[1] (MINA), [1] (RAG Survey)	Guarantees strict doctrinal coherence by forcing the model to lock onto an Act before a Section.	Two distinct ChromaDB collections; Stage 2 uses <code>where={"act_id": {"\$in": acts}}</code> .
Relevance Verification Gate	LLMs hallucinate case law when given irrelevant context.	[1], [1]	Fails safely rather than providing illegal advice.	Cosine similarity check between query and retrieved chunks before Gemma 4 prompt injection.
IRAC Prompting	Generative models struggle with sequential deductive reasoning.	[1] (Dehghani), [1]	Aligns the AI's output with professional legal standards, drastically reducing logical leaps.	Hardcoded Chain-of-Thought template in the system prompt.

Legal Dictionary Tool	General embeddings misinterpret historical legal terminology.	[1] (MINA), [1] (Yadav)	Indian law uses archaic Persian/Urdu terms (e.g., "Khasra", "Khatauni").	A static JSON key-value lookup tool triggered during the augmentation phase.
------------------------------	---	-------------------------	--	--

5. Novel Contributions

The research focused heavily on established NLP tasks and high-resource or localized setups. To bridge the gap for a production-ready Indian Legal Assistant, we introduce the following original engineering contributions:

- **Code-Mixed Language (Hinglish) Normalization:** Native embeddings fail on Romanized Hindi (e.g., "Mera landlord mujhe nikaal raha hai"). We will implement a lightweight transliteration and normalization layer using the AI4Bharat IndicTrans2 model *before* embedding the query, ensuring high-fidelity retrieval.
- **Indian Legal System Adaptation (BNS/BNSS):** We will specifically map the transition from the old IPC/CrPC to the new Bharatiya Nyaya Sanhita (BNS) and Bharatiya Nagarik Suraksha Sanhita (BNSS), ensuring the system provides updated 2024+ procedural guidance.
- **Legal Readiness Score:** An original UX metric. After a consultation, the system generates a "Readiness Score" (0-100%) indicating how prepared the citizen is to file a complaint (e.g., "You have the timeline and the grievance, but lack documentary evidence. Score: 60%").
- **Voice-First Interaction (Accessibility):** Integrating seamless push-to-talk using Bhashini APIs to cater to illiterate or semi-literate rural populations, bypassing the need for text input entirely.
- **Government Schemes Integration:** Many legal disputes for low-income citizens (e.g., land rights, domestic disputes) have corresponding state welfare relief. The system will uniquely cross-reference legal rights with available government schemes (e.g., NALSA free legal aid eligibility).
- **Procedural Workflow Guidance:** Instead of just outputting a legal conclusion, the system will generate a step-by-step flowchart (e.g., 1. Send Legal Notice -> 2. Wait 15 days -> 3. File in Consumer Forum).

6. Knowledge Base Design

Instead of dumping raw PDFs into a vector database, the legal corpus will be rigorously structured hierarchically to preserve legislative boundaries.

Hierarchy:Code/Law -> Part/Chapter -> Section/Article -> Clause/Sub-clause

Node Storage Strategy:

- **Level 1 (Acts/Codes):** Stored as LLM-generated semantic summaries (e.g., "The Bharatiya Nyaya Sanhita governs criminal offenses...").
- **Level 2 (Sections):** Stored as raw, chunked text.

Metadata Schema (Attached to every Section chunk):

- doc_id: Unique UUID
- act_title: e.g., "Bharatiya Nyaya Sanhita, 2023"
- act_id: e.g., "BNS_2023"
- chapter_num: e.g., "Chapter V"
- section_num: e.g., "Section 103"
- jurisdiction: "Union of India"
- effective_date: "01-07-2024"
- is_amended: Boolean
- keywords: ["murder", "culpable homicide", "life imprisonment"]

7. RAG Architecture

Our RAG architecture relies on a highly constrained, metadata-driven pipeline.

- **Chunking:** Semantic chunking strictly along natural legislative boundaries (Section/Clause level). No arbitrary token splitting (e.g., 512 tokens) that slices sentences mid-definition.
- **Embeddings:** Cohere embed-multilingual-v3.0 (as validated in [1]) to map multiple Indian languages into a shared latent space.
- **Vector Database:** ChromaDB, chosen for its fast metadata filtering capabilities.
- **Retrieval Flow:**
 1. *Hybrid Retrieval:* Keyword searches combined with dense vector similarity.
 2. *Hierarchical Filtering:* Retrieve top 3 Act summaries -> Extract act_ids -> Query Section database WHERE act_id IN [retrieved_acts].
- **Re-ranking:** Apply Cohere Re-ranker to order the retrieved sections based on exact relevance to the user's grievance.
- **Grounding & Citation:** The prompt forces Gemma 4 to cite the section_num and act_title metadata for every generated legal claim.

8. Multilingual Architecture

The Challenge: India has 22 official languages and widespread code-mixing

(Hinglish). **Translation Strategy Decision:** *Native Multilingual Retrieval + English Generation + Edge Translation.*

1. **Input Phase:** User speaks/types in Hinglish or regional language.
2. **Normalization:** Hinglish is transliterated to standard Hindi using a lightweight dictionary/model.
3. **Retrieval Phase (No Translation):** We do *not* translate the query to English for retrieval. We use Cohere's Multilingual Embeddings, which natively map Hindi, Bengali, Tamil, and English to the same vector space, directly matching the regional query to the English legal

database [1].

4. **Generation Phase (Gemma 4):** The retrieved English statutes and the original regional query are passed to Gemma 4. Gemma 4 is prompted to perform its IRAC reasoning in English (for maximum logical fidelity) but output the final "Citizen Explanation" in the user's detected language.
5. **Speech Output:** The regional text is converted to voice via Bhashini TTS.

Justification: Translating legal queries before retrieval destroys nuanced legal semantics. Multilingual embeddings preserve intent. Reasoning in English ensures Gemma 4 uses its strongest cognitive capabilities, while translating the final output ensures accessibility.

9. Prompt Engineering

System Prompt (Persona & Constraints): "You are a highly expert, ethical AI Legal Assistant for Indian citizens. You provide legal information, not binding legal advice. You must base your answers STRICTLY on the provided retrieved statutory context. If the context is insufficient, you must state: 'I do not have enough information to answer this.' Do not use your parametric memory for case laws."

Developer Prompt (IRAC Enforcement): "Structure your response using the IRAC methodology:

1. **Issue:** Identify the core legal grievance.
2. **Rule:** State the exact retrieved laws and sections.
3. **Application:** Apply the user's specific facts to the rule.
4. **Conclusion & Next Steps:** Provide a clear, actionable summary and 3 procedural steps."

Clarification Prompt: "Analyze the user's query. Identify if any of the following critical legal variables are missing: Date of incident, Relationship between parties, Jurisdiction/State, Monetary value. If missing, generate a single, polite clarifying question. Do not generate a legal answer."

Citation Prompt: "Every factual legal claim must end with a citation enclosed in brackets, using ONLY the metadata provided in the context. Example:."

10. Safety

- **Hallucination Prevention:** The Relevance Verification Gate (Cross-encoder) blocks generation if retrieved context similarity is < 0.70 .
- **Unauthorized Legal Advice:** Hardcoded disclaimers appended to every output: *"This is AI-generated legal information, not professional legal counsel."*
- **Privacy Leaks:** PII (Names, Aadhar numbers, Phone numbers) detected via regex/spaCy and redacted *before* the query is sent to the LLM or embedding API.
- **Prompt Injection Defense:** A lightweight classifier inspects user inputs for injection commands (e.g., "Ignore previous instructions") and terminates the session if detected.
- **Human-in-the-Loop:** If the system detects high-stakes criminal liability (e.g., murder, sexual assault keywords), it immediately escalates the user to a human legal aid hotline [1], [1].

11. User Experience

The Interface:

- **Voice-First Chat Interface:** A prominent microphone button takes precedence over the text box.
- **Visual Provenance:** When the AI cites a law, the text is hyperlinked. Clicking it opens a side-panel displaying the exact original text of the statute from the database, building immense trust [1], [1].
- **Procedural Tracker:** A visual progress bar at the top of the chat showing the user where they are in the legal process (e.g., *Notice* -> *FIR* -> *Magistrate* -> *Resolution*).
- **Confidence Indicators:** A subtle traffic-light system (Green/Yellow) indicating the AI's confidence based on the relevance score of the retrieved documents.

12. Technology Stack

- **Frontend:** React.js + TailwindCSS (Fast, responsive, supports mobile-first design).
- **Backend:** FastAPI (Python) (High performance, async support for concurrent LLM/DB calls).
- **Agent Framework:** LangGraph (Essential for maintaining conversational memory and orchestrating complex state-machine routing [1]).
- **Database:** PostgreSQL (User sessions, chat history) + Redis (Fast caching for dialogue state).
- **Vector DB:** ChromaDB (Lightweight, optimized for the strict metadata filtering required by Two-Stage RAG [1]).
- **Embeddings:** Cohere embed-multilingual-v3.0 (Proven superiority in handling complex South Asian script morphologies without translation [1]).
- **Inference (LLM):** Gemma 4 (via Google API or locally hosted via vLLM for the Hackathon MVP, mandated by problem statement).
- **Translation/Speech:** Bhashini API / SeamlessM4T (Optimized specifically for Indian regional languages and dialects).
- **OCR:** EasyOCR + PyMuPDF (For extracting text from user-uploaded legal notices).

13. Development Roadmap

Phase 1: Hackathon MVP (48-72 Hours)

- Ingest the Bharatiya Nyaya Sanhita (BNS) and Bharatiya Nagarik Suraksha Sanhita (BNSS) into ChromaDB.
- Implement the Two-Stage Hierarchical RAG pipeline.
- Build the LangGraph Orchestrator and basic IRAC prompting with Gemma 4.
- Deploy a React frontend with basic English/Hindi text support.

Phase 2: Version 2 (Post-Hackathon)

- Integrate Voice-to-Text (Bhashini) and Hinglish normalization.
- Add the Legal Dictionary tool for archaic terminology.

- Implement visual provenance mapping in the UI.

Phase 3: Version 3 & Future Scope

- Implement Knowledge Graph RAG (GraphRAG) to natively map relational dependencies and amendments between different legal statutes [1].
- Add automated legal drafting (e.g., generating court-ready PDF petitions based on the conversation).
- Full integration with state legal aid APIs for direct human-lawyer handoff.

14. Evaluation

We will abandon meaningless NLP metrics like BLEU/ROUGE and evaluate based on ecological legal validity [1], [1].

- **Accuracy & Legal Correctness:** Blind benchmarking against historical Indian Bar Examination questions using Bloom's Cognitive Taxonomy [1].
- **Grounding & Hallucination Rate:** Evaluated using the **RAGAS framework** [1], explicitly scoring *Context Precision*, *Context Recall*, and *Faithfulness*.
- **Multilingual Performance:** Human evaluation of semantic preservation when translating legal terms from English to Hindi/regional languages.
- **Latency:** Must maintain < 3 seconds time-to-first-token for voice interfaces to feel natural.

15. Innovation

Why is this different from existing tools? Existing platforms (like ChatGPT or basic legal wrappers) rely on parametric memory, which leads to catastrophic hallucinations of fake case laws [1]. Furthermore, standard RAG systems suffer from "flattening," where maritime law might be applied to a domestic dispute due to vector similarity [1].

Our system innovates by:

1. **Hierarchical Strictness:** Using a Two-Stage Act->Section filter that mathematically guarantees doctrinal coherence.
2. **Citizen-Centric, Not Lawyer-Centric:** Focusing entirely on statutory rights, procedural steps, and government schemes rather than complex case-law precedent analysis.
3. **Overcoming the Language Barrier:** Natively processing Code-Mixed Hinglish and voice inputs, acknowledging the reality of India's digital literacy landscape.

16. Final Engineering Blueprint

Compon ent	Purpose	Researc h Source	Our Improve ment (Original)	Priority	MVP	Future Scope

Orchestrator Agent	Routes queries, manages memory, bypasses RAG for chat.	MINA [1]	Added "Clarification" state to force user to provide missing case facts.	Critical	Yes	Multi-agent debate.
Hierarchical RAG	Prevents statutory cross-contamination during retrieval.	MINA [1], Surveys [1]	Tailored specifically to Indian Codes (Act -> Chapter -> Section metadata).	Critical	Yes	Knowledge Graphs.
Multilingual Embeddings	Enables native cross-lingual retrieval.	MINA [1]	Pre-processing layer to normalize Romanized Hinglish before embedding.	High	Yes	Fine-tuned Indic models.
IRAC Prompting	Enforces deductive legal reasoning format.	Dehghani [1]	Augmented with "Procedural Next Steps" for actionable citizen guidance.	Critical	Yes	Dynamic prompt generation.

Relevance Gate	Halts generation if retrieved context doesn't match query.	MINA [1], Surveys [1]	Triggers fallback to human legal aid if confidence drops below 0.7.	High	Yes	Adversarial self-check.
Legal Dictionary	Maps archaic terms to modern semantic equivalents.	MINA [1], Yadav [1]	Custom JSON mapping specifically for Persian/Urdu Indian land & police jargon.	Medium	No	Automated glossary updates.
Explainability (UI)	Builds user trust by showing exact legal sources.	Dehghani [1]	Visual provenance mapping connecting generated text directly to source PDF.	High	Yes	Interactive citation graphs.
Voice Interaction	Ensures accessibility for illiterate/rural citizens.	Original	Integration of Bhashini APIs for localized dialect speech-to	High	Yes	Real-time voice interruption.

			-text.			
Legal Readiness Score	Gamifies and quantifies user preparedness.	Original	Algorithm calculating completeness of user's facts/documents.	Low	No	Predictive case success rate.